

**Esquema de Esteganografía y Marca de Agua Robusta
Basado en ANS y STC**

Presentado por:

Michael Stiven Betancourt Gelves

Profesor:

Oswaldo Rojas Camacho

Lunes 06 de Julio



**Universidad Nacional de Colombia
Facultad de ingeniería
Teoría de la Información
2026**

1. Problema o necesidad

Hoy es muy fácil crear imágenes falsas. La Inteligencia Artificial generativa ha llenado la red de *deepfakes*, y por eso ya no basta con *ver para creer*. Lo que hace falta es una forma *matemática* de comprobar que una imagen es auténtica y de saber de dónde viene. Una manera de lograrlo es incrustar dentro de la propia imagen una *marca de agua invisible de procedencia*: una firma que un verificador pueda leer más tarde para confirmar el origen del archivo.

El problema es que esa firma tiene que *sobrevivir*. Cuando subimos una foto a una red social, la plataforma la *recomprime* (normalmente a JPEG) y, sobre todo, la *reescala* a un tamaño más pequeño. La esteganografía clásica esconde los bits en el dominio espacial o en bloques de 8×8 de la Transformada Discreta del Coseno (DCT); ambas operaciones *desincronizan* esa retícula de 8×8 y borran el mensaje oculto. Basta una edición mínima para que la marca desaparezca.

Visto con las gafas de la Teoría de la Información, este es un problema de **transmitir un mensaje por un canal con ruido**. Si $\mathbf{x}$ es la imagen portadora, $\mathbf{m}$ el mensaje que queremos ocultar y $C(\cdot)$ el operador del canal (reescalar y después recomprimir), necesitamos un codificador E y un decodificador D tales que

$$D(C(E(\mathbf{x}, \mathbf{m}))) = \mathbf{m} \quad (1)$$

se cumpla con *alta probabilidad*, y todo esto manteniendo una distorsión visual pequeña entre la imagen original $\mathbf{x}$ y la imagen marcada $E(\mathbf{x}, \mathbf{m})$. En pocas palabras: el mensaje debe ser *invisible* para el ojo y a la vez *recuperable* después de que la imagen pase por las manos de la red social.

¿Por qué es importante? Una marca de procedencia que resista el paso por Facebook, WhatsApp o Pinterest sirve como herramienta anti-*deepfake*: permite firmar una imagen auténtica y verificar esa firma aunque la imagen haya sido reescalada y recomprimida por el camino. Es una pieza pequeña pero concreta dentro del problema, mucho más grande, de la confianza en los medios digitales.

2. Objetivos

Objetivo general

Diseñar e implementar un sistema capaz de ocultar un mensaje verificable dentro de una imagen de modo que permanezca imperceptible al ojo humano y, al mismo tiempo, sobreviva a la compresión y al reescalado que aplican las redes sociales, apoyándonos en herramientas clásicas de teoría de la información.

Objetivos específicos

1. Construir una *marca de agua robusta* que incruste un texto corto y verificable y sobreviva la cadena completa de reescalado y recompresión de Facebook, WhatsApp y Pinterest, usando espectro ensanchado mejorado (ISS) sobre una malla de resolución fija.
2. Apoyar ese sistema en herramientas clásicas de teoría de la información —compresión entrópica (Brotli), corrección de errores (Reed-Solomon), el teorema de muestreo y el enmascaramiento perceptual— para lograr a la vez robustez e imperceptibilidad.

3. *Medir* de forma cuantitativa el compromiso entre robustez, imperceptibilidad y detectabilidad, con métricas objetivas (PSNR, SSIM, BER) y un pequeño banco de pruebas de esteganálisis.

3. Marco teórico

En esta sección explicamos, con calma, las ideas matemáticas que hacen falta para entender la solución. Cada una viene de un resultado clásico y bien conocido; aquí contamos la intuición y la fórmula esencial, y más adelante (Sección 5) mostramos cómo se usan dentro del sistema.

3.1. Entropía de Shannon y codificación de fuente

La *entropía* mide cuánta información produce, en promedio, una fuente de datos. Para una fuente discreta X con distribución de probabilidad $p(x)$ se define como

$$H(X) = - \sum_x p(x) \log_2 p(x). \quad (2)$$

La intuición es sencilla: un símbolo muy probable “sorprende poco” y cuesta pocos bits; uno raro sorprende mucho y cuesta más. La entropía es el promedio de esa sorpresa, medido en bits por símbolo. El *teorema de codificación de fuente* de Shannon [1] dice que ningún método sin pérdida puede usar, en promedio, menos de $H(X)$ bits por símbolo, y que ese límite se puede alcanzar tan de cerca como se quiera.

¿Por qué nos importa esto para ocultar datos? Porque en esteganografía **cada bit que escondemos es una modificación en la imagen**. Si comprimimos el mensaje antes de incrustarlo, tenemos menos bits que ocultar; y menos bits significan menos cambios (menos distorsión) y más margen para la corrección de errores. Comprimir no es un lujo: es lo que hace posible que el sistema sea a la vez invisible y robusto.

3.2. Codificadores entrópicos: Huffman, Lempel-Ziv y ANS

Para acercarnos al límite $H(X)$ usamos codificación entrópica. Conviene tener presentes tres ideas:

- **Códigos de Huffman** [2]. Construyen, con un árbol binario, el mejor código de prefijo posible *cuando las longitudes son enteras*: los símbolos frecuentes reciben códigos cortos y los raros, códigos largos. Son rápidos y óptimos dentro de esa familia, aunque pueden desperdiciar hasta cerca de 1 bit por símbolo por la restricción de longitud entera.
- **Codificación por diccionario Lempel-Ziv (LZ77)** [3]. En vez de mirar símbolo por símbolo, sustituye subcadenas repetidas por una referencia (distancia, longitud) a una copia anterior. Así captura la redundancia *del contexto* (palabras y frases que se repiten) sin conocer de antemano la distribución. Es la base de compresores modernos como Brotli.
- **Sistemas Numéricos Asimétricos (ANS)** [5]. Representan todo el mensaje como un *único* número natural x que va creciendo. Para un símbolo s con frecuencia f_s y frecuencia acumulada c_s , sobre un total $M = 2^b$, el estado avanza según

$$x' = \left\lfloor \frac{x}{f_s} \right\rfloor M + (x \bmod f_s) + c_s. \quad (3)$$

Lo bonito de ANS es que junta lo mejor de dos mundos: la *velocidad* de Huffman y la *eficiencia* de la codificación aritmética [4], quedando muy cerca del límite de entropía. La operación de la Ecuación (3) es exactamente invertible, así que el proceso es sin pérdida.

3.3. La Transformada Discreta del Coseno (DCT)

Casi todo el trabajo ocurre en el dominio de la *frecuencia*, y la herramienta para llegar allí es la DCT. La DCT-II bidimensional y ortonormal de un bloque $f(x, y)$ de tamaño $N \times N$ es

$$F(u, v) = \alpha(u) \alpha(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos \left[\frac{\pi(2x+1)u}{2N} \right] \cos \left[\frac{\pi(2y+1)v}{2N} \right], \quad (4)$$

con $\alpha(0) = \sqrt{1/N}$ y $\alpha(k) = \sqrt{2/N}$ para $k > 0$. Esos factores hacen que la transformada sea *ortonormal*, es decir, que conserve la energía (teorema de Parseval): una distorsión medida en el dominio DCT equivale a la misma distorsión medida en los píxeles. La DCT fue introducida por Ahmed, Natarajan y Rao [6] y tiene una propiedad muy útil: concentra casi toda la energía de una imagen natural en unos pocos coeficientes de baja frecuencia. Por eso conviene esconder información en la *banda media*: la componente continua (DC) es demasiado visible y las frecuencias más altas son las primeras que JPEG destruye, así que el punto justo está en el medio.

3.4. El sistema visual humano y el enmascaramiento por contraste

Nuestro ojo no es igual de sensible en todas partes: tolera mucho más ruido en las zonas con textura (hierba, follaje, arena) que en las zonas lisas (un cielo despejado, la piel). A este fenómeno se le llama *enmascaramiento por contraste*, y los modelos perceptuales sobre la DCT [8] lo formalizan con umbrales de visibilidad que dependen del contenido local. La idea que aprovechamos es simple: si vamos a modificar la imagen, hagámoslo donde el ojo menos lo note, es decir, en la textura.

3.5. Códigos Reed-Solomon (corrección de errores)

Aunque protejamos bien el mensaje, el canal siempre introducirá algún error. Para repararlo envolvemos los datos con un código Reed-Solomon [9]. Un código RS(n, k) sobre el cuerpo finito $\text{GF}(2^8)$ interpreta k bytes de datos como los coeficientes de un polinomio y lo evalúa en $n = 255$ puntos, añadiendo $n - k$ bytes de paridad. Por la cota de distancia mínima (es un código MDS, con distancia $d = n - k + 1$ [10]) puede corregir hasta

$$t = \left\lfloor \frac{n-k}{2} \right\rfloor \quad (5)$$

bytes erróneos por bloque. Lo mejor es que trabaja con *bytes*: un byte dañado cuenta como un solo error sin importar cuántos de sus ocho bits fallaron. Esto lo hace ideal para errores en *ráfaga*, que es justo el tipo de error que produce nuestro sistema.

3.6. Verificación por redundancia cíclica (CRC)

A la cabecera de cada contenedor le añadimos una comprobación CRC-16 [11]. El mensaje se ve como un polinomio sobre $\text{GF}(2)$ y se toma el residuo módulo el polinomio generador

$g(x) = x^{16} + x^{12} + x^5 + 1$ (estándar CCITT). El CRC no *corrige* nada, solo *detecta* con altísima probabilidad si la cabecera está dañada. Su papel es evitar falsos positivos: una imagen sin marca (o demasiado degradada) falla el CRC y el sistema responde honestamente “no hay marca” en lugar de devolver basura.

3.7. Codificación de mínima distorsión: la herencia ANS-STC

El nombre del proyecto recuerda su punto de partida teórico. Una línea muy potente de la esteganografía moderna combina un codificador entrópico como ANS (los codificadores que vimos arriba) con los **Códigos de Síndrome-Trellis (STC)** [14]: dado un mensaje $\mathbf{m}$, el STC busca el estego $\mathbf{y}$ que cumple $\mathbf{H}\mathbf{y} = \mathbf{m}$ *cambiando lo menos posible* el portador. Ese “lo menos posible” se mide con una función de costo perceptual como J-UNIWARD [17] (construida sobre wavelets de Daubechies [7]) y se resuelve de forma óptima con el algoritmo de Viterbi [15]. Es un enfoque excelente para ocultar *mucha* información resistiendo la recompresión, pero tiene un talón de Aquiles: depende de la retícula de bloques 8×8 y por eso *muere cuando la imagen se reescala*. Como nuestro objetivo es sobrevivir al reescalado de las redes sociales, tomamos de aquí el principio rector —*minimizar los cambios en el portador*— pero lo llevamos a un esquema distinto, el espectro ensanchado, que sí tolera el cambio de tamaño.

3.8. Espectro ensanchado y el teorema de muestreo

Dos ideas más sostienen el sistema:

- **Espectro ensanchado.** En vez de poner un bit en un solo coeficiente, lo *repartimos* sobre muchos, con un patrón pseudoaleatorio de signos. Al promediar en la detección se obtiene una gran “ganancia de procesamiento” que hace la lectura muy robusta al ruido [12]. Su versión mejorada (ISS) [13] se explica en detalle en la Sección 5.
- **Teorema de muestreo de Nyquist-Shannon** [18, 19]. Una señal solo se puede reconstruir si se muestrea a más del *doble* de su frecuencia más alta. Trasladado a imágenes: al reescalar a W píxeles solo sobreviven las frecuencias por debajo de $W/2$; todo lo que esté por encima lo elimina el filtro anti-*aliasing*. Este resultado, que parece muy abstracto, nos dará una regla práctica muy concreta.

4. Metodología

El proyecto se desarrolló en **Python**, apoyándonos en bibliotecas científicas de uso estándar: NumPy y SciPy para el álgebra y las transformadas, Pillow para leer y escribir imágenes, reedsolo para la corrección de errores Reed-Solomon, brotli para la compresión de texto y customtkinter para la interfaz gráfica. No se usó ningún modelo de aprendizaje automático de “caja negra”: todo es matemática explícita y reproducible.

Ante el problema planteado, el sistema implementado es una **marca de agua robusta** cuya tubería, de principio a fin, es

texto $\xrightarrow{\text{Brotli}}$ comprimido $\xrightarrow{\text{Reed-Solomon}}$ protegido $\xrightarrow{\text{ISS / malla canónica}}$ imagen marcada.

La idea es fácil de enunciar y difícil de lograr: comprimimos el texto para tener menos bits que ocultar, los protegemos con paridad para tolerar errores, y los repartimos por espectro ensanchado sobre una malla de resolución fija para que el reescalado no los borre.

El desarrollo se organizó en módulos independientes y bien separados (uno por responsabilidad: marca de agua, esteganálisis, simulador de canal y utilidades compartidas), lo que facilitó probar cada pieza por separado. Para validar el sistema escribimos una batería de **94 pruebas automáticas** que comprueban, entre otras cosas, que el ciclo marcar/verificar es exacto y que una imagen sin marca se reporta correctamente como tal. Además construimos un *simulador de canal* que reescala y recomprime la imagen a JPEG con distintas calidades y mide cuánto sobrevive el mensaje; esta herramienta nos permitió elegir los parámetros de forma empírica en lugar de a ciegas.

5. Implementación y desarrollo de la solución

Aquí está el corazón del proyecto. Explicamos el diseño del sistema, el modelo matemático que hay detrás y cómo encajan las piezas.

Resincronización a una resolución canónica

La esteganografía por bloques muere ante el reescalado porque pierde la retícula de 8×8 . La idea central de nuestra marca es *renunciar* a esa retícula: tanto el insertor como el extractor llevan primero la luminancia Y a una malla fija $S \times S$ (con $S = 1152$) mediante remuestreo,

$$Y_c = \mathcal{R}_{S \times S}(Y), \quad (6)$$

y trabajan sobre una *única* DCT global de Y_c . Da igual a qué tamaño arbitrario reescale la plataforma: el extractor devuelve la imagen a esa misma malla canónica antes de decodificar, de modo que la retícula de incrustación nunca se pierde. Solo la *perturbación* de la marca se remuestrea de vuelta al tamaño original, así que la imagen conserva su aspecto y se ve limpia.

Espectro ensanchado mejorado (ISS)

La información se incrusta con Espectro Ensanchado Mejorado [13]. Sea G_i el grupo de $L = |G_i|$ coeficientes asignados al bit b_i , con signos $s_c \in \{-1, +1\}$ y amplitud α . Definimos el objetivo $\tau_i = \alpha(2b_i - 1)$ y la proyección previa

$$p_i = \frac{1}{\sqrt{L}} \sum_{c \in G_i} s_c D(c). \quad (7)$$

La regla de inserción actualiza cada coeficiente como

$$D'(c) = D(c) + \frac{\tau_i - p_i}{\sqrt{L}} s_c. \quad (8)$$

Aquí ocurre la magia. Como $s_c^2 = 1$, si calculamos la proyección *después* de insertar obtenemos exactamente

$$p'_i = \frac{1}{\sqrt{L}} \sum_{c \in G_i} s_c D'(c) = p_i + \frac{\tau_i - p_i}{L} \sum_{c \in G_i} s_c^2 = \tau_i, \quad (9)$$

es decir, la correlación queda fijada en $\pm\alpha$ y el término del portador p_i *desaparece* (por eso se llama “mejorado”). La detección es ciega y por *signo*:

$$\hat{b}_i = \mathbb{K} \left[\sum_{c \in G_i} s_c \tilde{D}(c) > 0 \right], \quad (10)$$

donde $\tilde{D}$ es el espectro recibido tras el canal. Al depender solo del signo, el extractor *no necesita conocer* la amplitud α , y la gran ganancia de procesamiento del espectro ensanchado hace la decisión muy resistente al ruido de la compresión.

Máscara perceptual y fuerza adaptativa

Dos ajustes finos mejoran mucho el resultado. Primero, una *máscara perceptual* multiplica la perturbación por la actividad local para concentrarla en la textura y retirarla de las zonas lisas. Con la varianza local en una ventana $w \times w$,

$$a(x, y) = \sqrt{\text{Var}_w[Y_c](x, y)}, \quad \text{mask}(x, y) = \text{clip}\left(\frac{a(x, y)}{\bar{a}}, 0.72, 3.0\right), \quad (11)$$

donde el suelo $0.72 > 0$ garantiza que hasta un cielo liso conserve marca suficiente para sobrevivir descargas pequeñas. Segundo, una *fuerza adaptativa*: la fiabilidad de la detección crece como $\sqrt{L}$ (coeficientes por bit), así que un mensaje largo, que tiene menos coeficientes por bit, sube su amplitud automáticamente para no perder margen,

$$\alpha_{\text{carga}} = \alpha \cdot \min\left(\beta_{\text{máx}}, \sqrt{\frac{C_{\text{ref}}}{L}}\right), \quad L = \frac{N - N_{\text{cab}}}{n_{\text{bits}}}, \quad (12)$$

con $C_{\text{ref}} = 400$ y tope $\beta_{\text{máx}} = 1.60$. Como la detección es por signo, este ajuste es transparente para el extractor: los mensajes cortos se insertan en voz baja (alta calidad) y los largos suben la voz lo justo para mantener el mismo margen de canal.

La cota de resolución de trabajo (aplicación del teorema de muestreo)

Aquí es donde el teorema de muestreo se vuelve práctico. Insertar sobre un original enorme obliga a la marca a atravesar una razón de remuestreo muy grande (canónico $\rightarrow$ nativo al insertar, y nativo $\rightarrow$ plataforma al descargar); la composición de dos filtros de interpolación imperfectos atenúa la banda media y hace fallar la recuperación. La solución es acotar el tamaño de trabajo: todo original cuyo lado mayor supere $S_{\text{máx}} = 2048$ se reduce a ese tamaño antes de insertar. Esto acota la razón de remuestreo y, además, coincide con lo que las plataformas conservan de todos modos. Fue la corrección clave para las fotos de alta resolución.

El contenedor: cabecera, compresión y corrección

La carga final es texto $\rightarrow$ Brotli $\rightarrow$ Reed-Solomon, precedida de una cabecera ultra-redundante de 64 bits con los campos MAGIC|LEN|NSYM|CRC-16|FLAGS. La cabecera se decodifica primero y anuncia cuántos bits de carga leer, de modo que los mensajes cortos usen menos bits (mayor calidad y margen). El texto se comprime con Brotli solo si eso reduce el tamaño; por eso el límite de 512 bytes es sobre el tamaño *comprimido*, y en la decodificación se acota el tamaño expandido para prevenir un ataque de descompresión (“bomba”).

6. Demostración

El sistema se puede usar de tres formas: una interfaz gráfica (python app.py), una línea de comandos (cli.py) y un ejecutable autónomo generado con PyInstaller que funciona sin tener Python instalado. La línea de comandos resume todo el sistema en unos pocos comandos:

```
# Marcar una imagen con un texto (hasta 512 bytes comprimidos)
python cli.py watermark --cover foto.jpg --text "firma" --out m.png

# Leer la marca (incluso de una imagen bajada de una red social)
python cli.py verify --stego descargada.jpg

# Medir la detectabilidad (esteganalisis)
python cli.py steganalysis --scheme watermark --images ./fotos
```

Al marcar una imagen, la consola informa del tamaño de la carga y del PSNR obtenido. El comando `verify` lee la marca incluso a partir de una imagen descargada de una red social, y responde “watermark found and CRC-valid” junto con el texto recuperado. La Sección 7 muestra estas salidas reales.

7. Resultados y análisis

7.1. ¿Funcionó?

Sí. Incrustar el texto `authentic:mike-2026` (19 bytes) da un PSNR de **38.45** dB, y el `verify` recupera el texto exacto con el CRC válido. En pruebas fieles a las tuberías de Facebook, WhatsApp y Pinterest (reducción a 2048/1600/736/1024 px más JPEG de calidad 60–85 con submuestreo de croma), una carga de 512 bytes se recupera con **cero errores de bit**, y sigue recuperándose incluso en descargas pequeñas del feed de Facebook hasta ~ 480 px.

7.2. Análisis de imperceptibilidad y detectabilidad

Para juzgar la calidad usamos cuatro métricas objetivas. El **PSNR**, sobre valores en $[0, 255]$, es

$$\text{PSNR} = 10 \log_{10} \frac{255^2}{\text{MSE}}, \quad \text{MSE} = \frac{1}{MN} \sum_{x,y} (I(x,y) - \hat{I}(x,y))^2, \quad (13)$$

y garantiza la invisibilidad (~ 38 dB para mensajes cortos, ~ 31 dB cerca del máximo). El **SSIM** [20] compara luminancia, contraste y estructura en ventanas locales,

$$\text{SSIM}(a,b) = \frac{(2\mu_a\mu_b + c_1)(2\sigma_{ab} + c_2)}{(\mu_a^2 + \mu_b^2 + c_1)(\sigma_a^2 + \sigma_b^2 + c_2)}, \quad (14)$$

y se acerca más a la percepción humana que el simple error cuadrático. El **BER** (tasa de error de bit) debe caer por debajo de la capacidad correctora del Reed-Solomon para que la reconstrucción sea exacta. Por último, la **divergencia de Kullback-Leibler** [21] mide la seguridad esteganográfica,

$$D_{\text{KL}}(P\|Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}, \quad (15)$$

y Cachin [22] define un esquema ε -seguro cuando $D_{\text{KL}}(P\|Q) \leq \varepsilon$; minimizar el número de cambios (lo que persiguen tanto la compresión como la máscara perceptual) reduce esta divergencia.

Para *medir* la detectabilidad incluimos un banco de pruebas de esteganálisis que extrae características **SPAM** de 686 dimensiones (co-ocurrencias de residuos de ruido [23]), entrena un discriminante lineal de Fisher y reporta el error del esteganalista bajo validación cruzada,

$$P_E = \min_t \frac{1}{2} (P_{\text{FA}}(t) + P_{\text{MD}}(t)), \quad (16)$$

donde $P_E \approx 0.5$ significa indetectable y $P_E \approx 0$ significa detectable. El resultado (Tabla 1) es coherente con la naturaleza del sistema: nuestra marca de agua es totalmente detectable *a propósito*. No busca ser encubierta, sino *robusta e infalsificable*; su valor está en resistir el reescalado, no en esconderse.

Tabla 1: Detectabilidad de la marca medida con un detector ligero (esteganálisis).

Esquema	P_E	ROC AUC	Lectura
Marca de agua (ISS)	~ 0.00	~ 1.00	detectable, por diseño

8. Dificultades encontradas

Durante el desarrollo aparecieron varios problemas, y cada uno nos empujó hacia una idea concreta de la teoría:

- **El reescalado borraba la marca.** Al principio, cualquier cambio de tamaño producía un “header sync mismatch”. La causa era la dependencia de la retícula de bloques 8×8 . *Solución:* la resincronización a una malla canónica.
- **Fallos en fotos de alta resolución.** En originales muy grandes (3000–6000 px) la marca se perdía tras la descarga. Entender que era un problema de *razón de remuestreo* (teorema de muestreo) nos llevó a la solución: acotar el tamaño de trabajo a 2048 px antes de insertar.
- **Portadores lisos y descargas pequeñas.** En imágenes muy planas (cielo, un documento) la marca casi desaparecía tras una descarga agresiva. *Solución:* un suelo en la máscara perceptual (0.72) y la fuerza adaptativa, que suben la amplitud lo justo para mantener el margen del canal.
- **Falsos positivos.** Sin una comprobación, una imagen sin marca devolvía basura. *Solución:* una cabecera con CRC-16 que permite responder honestamente “no hay marca”.
- **Fragilidad de la cabecera.** Como la cabecera es la parte más crítica, la protegimos con repetición y voto por mayoría, de modo que tolera una tasa de error muy alta.

9. Conclusiones

1. **Se cumplió el objetivo.** Construimos una marca de agua robusta que sobrevive el reescalado y la recompresión de Facebook, WhatsApp y Pinterest sin perder la invisibilidad, recuperando el texto de forma exacta.
2. **Un principio lo une todo.** Minimizar el número de cambios en el portador —vía compresión entrópica, reparto por espectro ensanchado y enmascaramiento perceptual— maximiza al mismo tiempo la robustez y la imperceptibilidad. No son objetivos opuestos si se atacan por ese lado.
3. **La teoría clásica resolvió problemas reales.** El teorema de muestreo dejó de ser abstracto y se volvió una regla de una línea (acotar a 2048 px) que arregló el fallo de alta resolución; la codificación de canal (Reed-Solomon) y el CRC hicieron la lectura fiable y honesta.

4. **Aprendimos a medir, no solo a afirmar.** El simulador de canal y el banco de esteganálisis nos dieron números (P_E , BER, PSNR) para cuantificar el compromiso entre robustez, invisibilidad y detectabilidad, en lugar de suponerlo.
5. **Aporte.** El resultado es una base matemática sólida y reproducible para una marca de procedencia anti-*deepfake* que funciona en el mundo real de las redes sociales.

Bibliografía

- [1] C. E. Shannon, “A Mathematical Theory of Communication,” *Bell System Technical Journal*, vol. 27, pp. 379–423 y 623–656, 1948.
- [2] D. A. Huffman, “A Method for the Construction of Minimum-Redundancy Codes,” *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.
- [3] J. Ziv y A. Lempel, “A Universal Algorithm for Sequential Data Compression,” *IEEE Transactions on Information Theory*, vol. 23, no. 3, pp. 337–343, 1977.
- [4] J. Rissanen y G. G. Langdon, “Arithmetic Coding,” *IBM Journal of Research and Development*, vol. 23, no. 2, pp. 149–162, 1979.
- [5] J. Duda, “Asymmetric numeral systems: entropy coding combining speed of Huffman with compression rate of arithmetic coding,” *arXiv:1311.2540*, 2014.
- [6] N. Ahmed, T. Natarajan y K. R. Rao, “Discrete Cosine Transform,” *IEEE Transactions on Computers*, vol. C-23, no. 1, pp. 90–93, 1974.
- [7] I. Daubechies, *Ten Lectures on Wavelets*. Philadelphia: SIAM, 1992.
- [8] A. B. Watson, “DCT quantization matrices visually optimized for individual images,” en *Proc. SPIE Human Vision, Visual Processing, and Digital Display IV*, vol. 1913, pp. 202–216, 1993.
- [9] I. S. Reed y G. Solomon, “Polynomial Codes over Certain Finite Fields,” *Journal of the Society for Industrial and Applied Mathematics*, vol. 8, no. 2, pp. 300–304, 1960.
- [10] F. J. MacWilliams y N. J. A. Sloane, *The Theory of Error-Correcting Codes*. Amsterdam: North-Holland, 1977.
- [11] W. W. Peterson y D. T. Brown, “Cyclic Codes for Error Detection,” *Proceedings of the IRE*, vol. 49, no. 1, pp. 228–235, 1961.
- [12] I. J. Cox, J. Kilian, F. T. Leighton y T. Shamoan, “Secure Spread Spectrum Watermarking for Multimedia,” *IEEE Transactions on Image Processing*, vol. 6, no. 12, pp. 1673–1687, 1997.
- [13] H. S. Malvar y D. A. F. Florêncio, “Improved Spread Spectrum: A New Modulation Technique for Robust Watermarking,” *IEEE Transactions on Signal Processing*, vol. 51, no. 4, pp. 898–905, 2003.
- [14] T. Filler, J. Judas y J. Fridrich, “Minimizing Additive Distortion in Steganography Using Syndrome-Trellis Codes,” *IEEE Transactions on Information Forensics and Security*, vol. 6, no. 3, pp. 920–935, 2011.

- [15] A. J. Viterbi, “Error Bounds for Convolutional Codes and an Asymptotically Optimum Decoding Algorithm,” *IEEE Transactions on Information Theory*, vol. 13, no. 2, pp. 260–269, 1967.
- [16] B. Li, M. Wang, J. Huang y X. Li, “A New Cost Function for Spatial Image Steganography,” en *Proc. IEEE International Conference on Image Processing (ICIP)*, pp. 4206–4210, 2014.
- [17] V. Holub, J. Fridrich y T. Denemark, “Universal Distortion Function for Steganography in an Arbitrary Domain,” *EURASIP Journal on Information Security*, vol. 2014, no. 1, art. 1, 2014.
- [18] H. Nyquist, “Certain Topics in Telegraph Transmission Theory,” *Transactions of the AIEE*, vol. 47, no. 2, pp. 617–644, 1928.
- [19] C. E. Shannon, “Communication in the Presence of Noise,” *Proceedings of the IRE*, vol. 37, no. 1, pp. 10–21, 1949.
- [20] Z. Wang, A. C. Bovik, H. R. Sheikh y E. P. Simoncelli, “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [21] S. Kullback y R. A. Leibler, “On Information and Sufficiency,” *The Annals of Mathematical Statistics*, vol. 22, no. 1, pp. 79–86, 1951.
- [22] C. Cachin, “An Information-Theoretic Model for Steganography,” en *Information Hiding*, LNCS 1525, pp. 306–318, 1998.
- [23] J. Fridrich y J. Kodovský, “Rich Models for Steganalysis of Digital Images,” *IEEE Transactions on Information Forensics and Security*, vol. 7, no. 3, pp. 868–882, 2012.
- [24] J. Kodovský, J. Fridrich y V. Holub, “Ensemble Classifiers for Steganalysis of Digital Media,” *IEEE Transactions on Information Forensics and Security*, vol. 7, no. 2, pp. 432–444, 2012.
- [25] C. R. Harris *et al.*, “Array Programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020. Documentación oficial: <https://numpy.org>.
- [26] P. Virtanen *et al.*, “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020. Documentación oficial: <https://scipy.org>.
- [27] Bibliotecas de software empleadas (repositorios oficiales): Pillow <https://python-pillow.org>, reedsolo <https://pypi.org/project/reedsolo>, Brotli <https://github.com/google/brotli>.

10. Anexos

A. Estructura del código

El sistema está dividido en módulos independientes, uno por responsabilidad:

Módulo	Responsabilidad
robust_watermark	marca de agua: ISS, malla canónica y enmascaramiento
steganalysis	banco de pruebas SPAM + Fisher (detectabilidad)
channel_simulator	modelo de supervivencia a reescalado y JPEG
config	parámetros compartidos del sistema

B. Parámetros principales del sistema

- **Compresión:** Brotli en modo texto (diccionario incorporado).
- **Corrección de errores:** Reed-Solomon con $n - k = 32$ bytes de paridad.
- **Malla canónica:** $S = 1152$ y DCT global sobre la luminancia.
- **Espectro ensanchado:** banda de frecuencias $[8, 400)$, amplitud base $\alpha = 100$.
- **Enmascaramiento y muestreo:** máscara perceptual recortada a $[0.72, 3.0]$ y cota de trabajo $S_{\text{máx}} = 2048$ px.
- **Capacidad:** hasta 512 bytes *comprimidos* por imagen.

C. Validación

El proyecto incluye **94 pruebas automáticas** que verifican el ciclo marcar/verificar, la robustez frente al reescalado y la recompresión JPEG, y que una imagen sin marca se reporte correctamente como tal. Puede ejecutarse con `python -m pytest -q`.

D. Manual de uso rápido

```
python app.py                                # interfaz grafica
python cli.py watermark --cover C --text T --out M # marcar
python cli.py verify --stego M                  # leer marca
python cli.py steganalysis --scheme watermark    # detectab.
```